

AGENTIC GTM & REVENUE OPERATIONS PLATFORM

Guia completo de arquitetura, stack e práticas de AI Engineering

Rebuild code-first do domínio de outbound / RevOps já validado em experiência profissional, com foco em portfólio para Applied AI Engineer, Agentic AI Engineer e AI Engineer.

Objetivo central

Construir um sistema agentic production-style que prove, em um único projeto público, domínio de Python, FastAPI, LangGraph, RAG, tool calling, MCP, evals, observabilidade, segurança, HITL, Redis/queues/workers, CI/CD, deploy versionado e rollback - sem depender de low-code como orquestrador principal.

Versão 1.0 | 25 de agosto de 2026

Documento de implementação e portfólio

Sumário executivo

- 1. Visão e problema de negócio
- 2. Escopo funcional e jornadas principais
- 3. Arquitetura alvo e responsabilidades por camada
- 4. Stack recomendada
- 5. Modelo de dados e estado do agente
- 6. Fluxos agentic e tool execution
- 7. Context engineering, RAG e memória
- 8. Segurança, governança e human-in-the-loop
- 9. Reliability, async processing e escala
- 10. Observabilidade, evals e LLMOps
- 11. CI/CD, versionamento e rollback
- 12. Roadmap de implementação
- 13. Critérios de aceite e definição de pronto
- 14. Deploy público, feedback e entregáveis de portfólio
- 15. Matriz completa de conceitos de AI Engineering aplicados

Princípio de escopo

Este projeto não deve virar um CRM genérico. O produto existe para demonstrar AI Engineering aplicado a um problema que você já conhece profundamente: priorização de contas, pesquisa, scoring, personalização, follow-up, execução segura de ações e atribuição de resultado em GTM/RevOps.

1. Visão e problema de negócio

1.1 Origem do projeto

O projeto parte de um domínio já trabalhado em produção: um sistema de cold outreach B2B com IA para a SociallyMe. Na implementação original, o processo cobria intake, deduplicação, scoring, personalização, envio via Instantly, eventos, classificação de replies e monitoramento operacional. O sistema processava 100 leads em menos de 5 minutos e filtrava aproximadamente 35% antes do outreach por ICP scoring.

O objetivo agora é reconstruir o domínio do zero, code-first, sem usar n8n como núcleo de orquestração. A nova versão não copia dados de cliente: o demo público deve usar dados sintéticos ou anonimizados e permissões próprias de ambiente de demonstração.

1.2 Problema de negócio

- Times de Sales/RevOps gastam tempo consolidando dados de leads e contas, pesquisando contexto, priorizando oportunidades e criando follow-ups.
- Automação simples tende a ser frágil em tarefas que exigem interpretação, contexto e decisões condicionais.
- Agentes com autonomia irrestrita introduzem risco: podem usar tools erradas, produzir argumentos inválidos, enviar mensagens inadequadas ou alterar dados sem rastreabilidade.
- Sistemas de IA sem evals, tracing e versionamento são difíceis de operar e melhorar com segurança.

1.3 Proposta de valor

Solução

Uma plataforma Agentic GTM / Revenue Operations que lê o estado comercial, recupera contexto, pesquisa contas, prioriza leads, recomenda ações, gera follow-ups, pede aprovação quando necessário, executa tools reais e registra tudo em um audit trail observável e avaliável.

1.4 Exemplos de comandos do usuário

"Find the highest-priority accounts that need attention today, research them and prepare personalized follow-ups."

"Which campaigns are generating the highest-quality pipeline?"

"Review replies from the last 24 hours, classify intent and create the appropriate next steps."

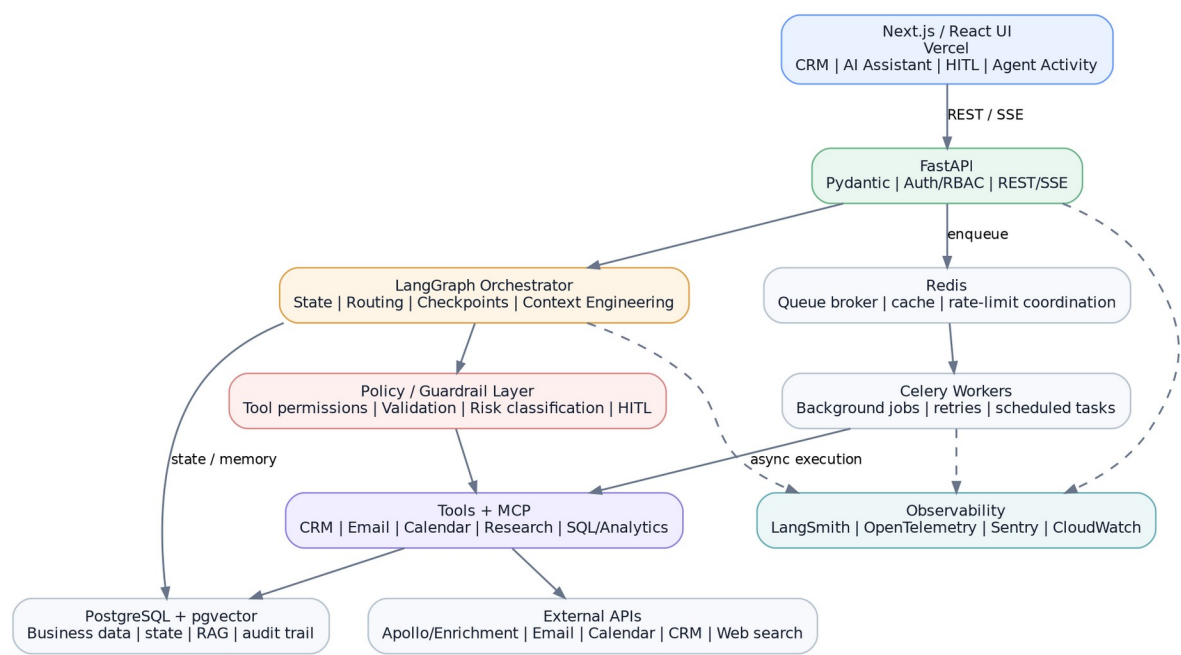
2. Escopo funcional e jornadas principais

Capacidade	Comportamento esperado	Sinal de portfólio
CRM / RevOps data model	Contacts, accounts, opportunities, campaigns, touchpoints, tasks e interactions relacionados por IDs estáveis.	SQL, modelagem, attribution, business state
Lead/account ingestion	Importação por CSV/API e enrichment assíncrono.	APIs, async Python, queues/workers
Deduplication	Normalize -> match -> merge -> master record -> preserve history.	Data integrity, idempotência vs dedup
ICP + priority scoring	Fit + intent + engagement, structured output e evidências.	LLM reasoning controlado, Pydantic
Research	Consulta de fontes externas permitidas e síntese com citations/metadata internas.	Tool calling, context engineering
RAG	ICP, produto, playbooks, brand voice e políticas comerciais.	Embeddings, pgvector, retrieval evals
Personalized outreach	Geração baseada em contexto real e knowledge base.	Prompt/context engineering
Human approval	Approve/Edit/Reject antes de ações externas de maior risco.	HITL, checkpoints, resumability
Email/calendar tools	Ações reais no modo autenticado; sandbox/mock no demo público.	Tool use, least privilege, security
Reply intelligence	Classificação estruturada + recommended next action.	Structured outputs, evals
Agent Activity	Exibir steps, tools, inputs/outputs, duração, aprovação e resultado.	Auditability, observability
Campaign analytics	Relacionar source/campaign -> contact/account -> opportunity -> revenue.	Attribution, SQL, BI thinking
Feedback loop	Rating/comentário ligado ao agent_run; falhas alimentam eval dataset.	Online feedback, LLMOps

2.1 Fluxo principal de valor

```
User request
-> retrieve CRM context
-> retrieve relevant RAG knowledge
-> research/enrich if needed
-> score/prioritize
-> propose plan
-> validate structured action
-> apply policy + permissions
-> HITL if high risk / low confidence
-> execute tool
-> persist business state + audit trail
-> expose trace/metrics
-> collect feedback
```

3. Arquitetura alvo



3.1 Responsabilidades por camada

Camada	Responsabilidade
Next.js / Vercel	Interface pública: CRM, pipeline, AI Assistant, HITL, Agent Activity, feedback e dashboards.
FastAPI	API síncrona/streaming, autenticação, RBAC, validação de requests, endpoints de agent runs, health checks e enqueue de jobs.
LangGraph	Orquestração stateful, conditional routing, checkpoints, interrupts, retries de graph-level, context assembly e retomada após HITL.
Policy / Guardrails	Validação determinística, tool allowlists, autorização, classificação de risco, confidence thresholds e regras de negócio.
Tools / MCP	Contratos claros para leitura/escrita no CRM, email, calendário, pesquisa, analytics e integrações externas.
PostgreSQL + pgvector	Business data, state persistente, audit trail, RAG, feedback e metadados de evals.
Redis + Celery	Fila, jobs long-running, batch scoring, enrichment, scheduled follow-ups, safe retries e coordenação de rate limits.
Observability	Traces, spans, logs, métricas, erros, tokens, custo, latency, tool success e feedback.
AWS	Backend/container runtime, database, Redis, secrets, logs e escalabilidade final do portfólio.

Decisão arquitetural importante

Comece single-agent. Separe agentes apenas quando evals mostrarem ganho real em contexto, permissões, qualidade ou manutenção.

4. Stack recomendada

Categoria	Stack alvo	Por que está no projeto
Linguagem	Python 3.12+	Linguagem principal de Applied/Agentic AI Engineering e backend.
Backend	FastAPI, Pydantic v2, SQLAlchemy, Alembic, httpx, asyncio	APIs, schemas, async I/O, persistência e migrations.
Agent framework	LangGraph	Stateful agents, graph orchestration, checkpoints, interrupts e long-running flows.
AI ecosystem	LangChain (auxiliar)	Retrievers, integrations e abstrações úteis sem dominar o core.
Model gateway	LiteLLM ou camada própria	Multi-provider, model routing, fallback, custo e telemetria.
LLMs	OpenAI + Anthropic; Gemini opcional	Comparar modelos e praticar routing/fallback.
Database	PostgreSQL	Business state, audit, analytics e persistência.
Vector store	pgvector	RAG e memória semântica sem adicionar banco separado.
Queue/cache	Redis	Broker/cache/coordenação; não confundir com controle direto de concurrency.
Workers	Celery	Background jobs, retries, batch processing e scheduled work.
MCP	FastMCP / custom MCP server	Interoperabilidade de tools e contratos padronizados.
Frontend	Next.js, React, TypeScript, Tailwind, shadcn/ui	Demo profissional e experiência full-stack.
Streaming	SSE	Exibir progresso e eventos do agente sem manter WebSocket desnecessário.
Observability AI	LangSmith	Tracing de graph/model/tool e evals.
Observability app	OpenTelemetry, Sentry, CloudWatch	Tracing distribuído, erros e operação.
Testing	pytest, Ruff, mypy	Qualidade de código e type safety.
CI/CD	GitHub Actions	Tests, eval gates, build e deploy.
Containers	Docker + Docker Compose	Reprodutibilidade local/prod.
IaC	Terraform	Infraestrutura versionada e repetível.
Frontend deploy	Vercel	URL pública, preview deployments e demo.
Backend deploy	AWS ECS/Fargate + RDS + ElastiCache	Experiência cloud e arquitetura production-style.
Secrets	AWS Secrets Manager	Credenciais fora de código e escopo seguro.

4.1 O que não é obrigatório no primeiro projeto

- Kubernetes: útil no mercado, mas adiciona complexidade sem melhorar o aprendizado central neste projeto.
- Fine-tuning/PyTorch/vLLM: relevantes para trilhas de ML/AI infrastructure, não necessários para provar Applied/Agentic AI Engineering aqui.
- Três frameworks de agentes no mesmo core: use LangGraph profundamente. PydanticAI/OpenAI Agents SDK podem virar POCs separados depois.
- Neo4j ou vector DB dedicado: PostgreSQL + pgvector já cobre o problema inicial.

5. Modelo de dados e estado

Entidade	Campos/relacionamentos essenciais	Uso
organizations	id, name, plan/demo_mode	Multi-tenant boundary.
users	id, organization_id, role	RBAC, approvals, audit.
contacts	id, account_id, email, phone, title, source	Pessoa no buying group.
accounts	id, domain, company_name, fit signals	Empresa / unidade de priorização.
opportunities	id, account_id, stage, value, status	Pipeline e revenue attribution.
campaigns	id, source, channel, metadata	Origem comercial.
touchpoints	id, contact_id/account_id, campaign_id, type, timestamp	Engagement e attribution.
tasks	id, owner, due_at, status, linked entity	Next actions.
interactions	id, contact/account, channel, content/summary	Histórico operacional.
agent_runs	id, user_id, thread_id, graph_version, prompt_version, model, status, latency, cost	Observability + reproducibility.
agent_actions	run_id, tool, input, output, status, approved_by, executed_at	Audit trail.
approvals	action_id, decision, edited_payload, user_id, timestamp	HITL completo.
feedback	run_id, rating, comment, created_at	Online evaluation loop.
knowledge_documents/chunks	source, version, embedding, metadata	RAG.
eval_cases/eval_runs	dataset_version, input, expected, scores, model/prompt/graph versions	Offline evaluation e CI gates.

5.1 Deduplication

Regra mental
NORMALIZE -> MATCH -> MERGE -> MASTER RECORD -> PRESERVE HISTORY

- Contacts: email normalizado como identificador primário; telefone e combinação nome+account como sinais secundários.
- Accounts: domínio/website normalizado como identificador forte.
- Nunca perder source, campaign, channel e touchpoints ao consolidar registros.
- Idempotência evita reprocessamento indevido; deduplication resolve registros duplicados. São problemas diferentes.

6. Fluxos agentic e tool execution

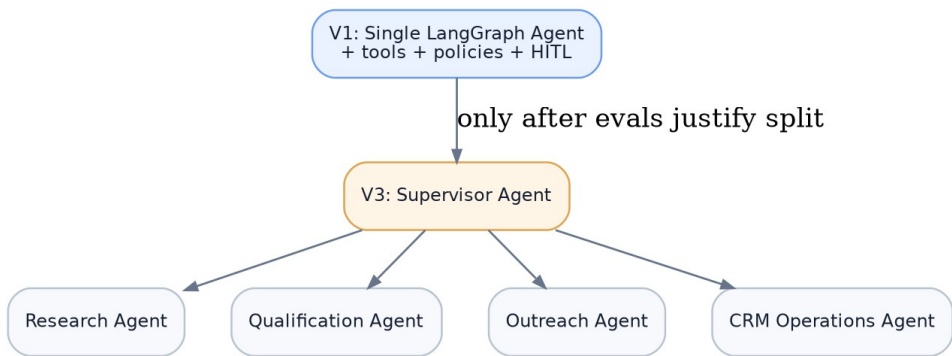
6.1 Single-agent V1

- O LangGraph Agent recebe a intenção do usuário e o estado relevante.
- Um router determina se a tarefa é leitura, análise, geração ou ação externa.
- Nodes recuperam CRM state, RAG context e dados externos quando necessário.
- O LLM produz saídas estruturadas; nenhuma tool de escrita recebe texto livre sem validação.
- A policy layer valida schema, permissões, regras de negócio e risco.
- Ações de risco alto ou baixa confiança param em HITL e retomam após decisão humana.

6.2 Tools iniciais

Tool	Tipo	Risco
get_account / search_accounts	read	baixo
get_contact / interaction_history	read	baixo
get_pipeline_metrics	read/analytics	baixo
research_company	external read	baixo-médio
create_task	write	médio
update_account/opportunity	write	médio
prepare_followup	generation	baixo
send_email	external write	alto
schedule_meeting	external write	alto

6.3 Evolução para multi-agent



- Research Agent: pesquisa externa, retrieval e síntese de evidências.
- Qualification Agent: fit/intent/engagement, scoring e explicação estruturada.
- Outreach Agent: mensagens e follow-up dentro de brand/policy constraints.
- CRM Operations Agent: updates, tasks, opportunities e ações administrativas.
- Supervisor: decide qual especialista executar; não ganha automaticamente todas as permissões das tools.

7. Context engineering, RAG e memória

7.1 Context engineering

- Construir o contexto por tarefa, não enviar o CRM inteiro ao modelo.
- Priorizar: objetivo atual, account/contact, opportunity, últimas interações, relevant touchpoints, policy, retrieved knowledge e outputs de tools anteriores.
- Definir token budget e truncation/summarization strategies para threads longas.
- Separar dados de instruções: conteúdo recuperado nunca deve ter autoridade para alterar system policy.
- Registrar quais chunks e quais entidades entraram no contexto para debugging e evals.

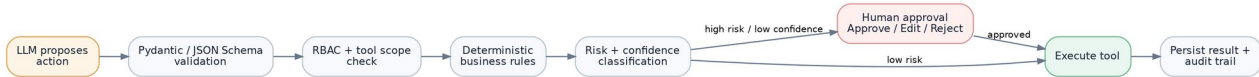
7.2 RAG

- Corpus: ICP, produto, pricing autorizado, playbooks, brand voice, objection handling, políticas de outreach e exemplos aprovados.
- Ingestion versionada com chunk metadata, source, document version e access scope.
- Embeddings em pgvector; adicionar hybrid retrieval e reranking na V2.
- Medir retrieval separadamente da generation: recall/relevance dos chunks antes de julgar a resposta final.

7.3 Tipos de memória

Memória	Onde fica	Exemplo
Thread / short-term state	LangGraph checkpoint	últimos passos, approvals pendentes, tool outputs
Business state	PostgreSQL	contacts, opportunities, tasks, interactions
Long-term knowledge	pgvector + document store	playbooks, ICP, product docs
User/org preferences	PostgreSQL com escopo/consentimento	brand voice, default policies

8. Segurança, governança e human-in-the-loop



8.1 Controles obrigatórios

- Least privilege: cada tool e credencial recebe somente os scopes necessários.
- Tool allowlist por agente, ambiente e role do usuário.
- Pydantic/JSON Schema para argumentos antes de execução.
- Deterministic business rules para limites, supressões, opt-out, domains bloqueados, quotas e estados inválidos.
- Risk classification para decidir autonomia vs HITL.
- Prompt injection defense: tratar conteúdo externo como dados não confiáveis; nunca permitir que RAG/web content redefina policy.
- PII redaction em traces/evals quando possível; segregação de tenant e secrets.
- Audit trail imutável ou append-only para ações sensíveis.

8.2 Human-in-the-loop

- Leitura/análise de baixo risco pode ser automática.
- Envio de email, bulk action, meeting scheduling e mudanças sensíveis exigem approval no MVP.
- UI deve mostrar exatamente: ação proposta, destinatário, payload, motivo, risco e evidências.
- Usuário pode Approve, Edit ou Reject; LangGraph retoma o checkpoint depois da decisão.

9. Reliability, async processing e escala

9.1 Padrões de reliability

Falha/Risco	Tratamento
Payload inválido	Pydantic validation + domain validation
API temporariamente indisponível	timeout + retry com exponential backoff + jitter
Rate limit	throttling, queueing, retry-after, quotas por provider
Duplicate request	idempotency key + safe upsert
Job falhou após retries	dead-letter / failed-job storage + safe reprocess
Dependência degradada	circuit breaker ou graceful degradation quando aplicável
Ação externa parcialmente executada	persistir status e reconciliation workflow
LLM output inválido	schema retry limitado + fallback path; nunca executar payload inválido

9.2 Sync vs async

Regra

SYNC = tarefa curta + resposta imediata. ASYNC = long-running, high-volume, external APIs, retries ou background processing.

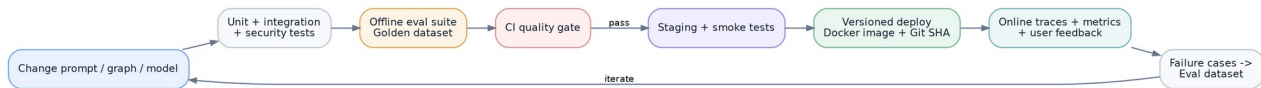
9.3 Filas e workers

- Redis funciona como broker/cache/coordenação. Concurrency é controlada pela configuração dos workers, não pelo Redis isoladamente.
- Celery workers para enrichment, batch scoring, scheduled follow-ups, bulk research, evals e integrações lentas.
- Escala horizontal: aumentar workers conforme queue length, latency e throughput.
- Persistir business state e execução relevante em PostgreSQL; não depender da fila como fonte de verdade.

9.4 Métricas operacionais

latency p50 / p95
throughput
queue length
worker utilization
API response time
job retry rate
tool failure rate
error rate
LLM tokens/run
cost/run
task success rate

10. Observabilidade, evals e LLMOps



10.1 Observabilidade

- LangSmith: graph/model/tool traces, inputs/outputs, latency, tokens e paths.
- OpenTelemetry: spans distribuídos entre FastAPI, LangGraph, workers e external APIs.
- Sentry: exceptions e erro de aplicação.
- CloudWatch: logs, container/infra metrics e alertas na AWS.
- Agent Activity: visão de produto do audit trail para recrutadores e usuários do demo.

10.2 Eval suite

Área	Métrica / teste
Lead scoring	agreement/accuracy contra golden labels e qualidade por score tier
Reply classification	accuracy/F1 por classe
Tool selection	tool-choice accuracy e unnecessary-tool rate
Tool arguments	schema validity + business-rule validity
Task success	percentual de runs que atingem objetivo
RAG retrieval	recall/relevance dos chunks recuperados
Groundedness	resposta sustentada pelo contexto disponível
Hallucination	unsupported-claim rate
Safety/policy	policy compliance e unsafe-action rate
Prompt injection	resistance suite com inputs adversariais
Performance	latency p50/p95, tokens/run, cost/run
Human feedback	rating, edit/reject rate, approval rate

10.3 Golden dataset e feedback-to-eval

```
evals/  
  datasets/  
    lead_scoring.jsonl  
    reply_classification.jsonl  
    tool_selection.jsonl  
    rag_questions.jsonl  
    adversarial_security.jsonl  
  scorers/  
    regression/  
    reports/
```

- Começar com exemplos sintéticos e/ou anonimizados. Nunca publicar dados confidenciais do cliente original.
- Runs ruins do demo público, após revisão, viram novos casos de regressão.
- LLM-as-judge pode complementar critérios subjetivos, mas deve ser combinado com métricas determinísticas quando possível.

11. CI/CD, versionamento e rollback

11.1 Pipeline de entrega

```
Pull Request
-> Ruff + mypy
-> pytest unit tests
-> integration tests
-> security/adversarial tests
-> offline eval suite
-> quality gate
-> Docker build (tag = Git SHA)
-> staging deploy
-> smoke tests
-> production deploy
-> online monitoring
-> rollback if quality/performance degrades
```

11.2 O que versionar

Artefato	Versão registrada
Código	Git commit SHA
Docker image	registry tag/digest
Graph	graph_version
Prompt	prompt_version
Model config	provider/model/temperature/limits
RAG corpus	document/index version
Eval dataset	dataset version/hash
Database schema	Alembic migration version

11.3 Rollback

- Voltar para imagem Docker anterior por SHA/digest.
- Restaurar prompt/graph/model config anterior sem depender de editar produção manualmente.
- Evitar migrations destrutivas; usar padrões backward-compatible quando possível.
- Registrar em agent_runs as versões usadas, permitindo reproduzir uma falha.

12. Roadmap de implementação

Fase	Objetivo	Entregáveis mínimos
0. Domain & design	Congelar problema, entidades, policies e eval plan.	ERD inicial; tool contracts; risk matrix; eval cases; repo skeleton.
1. Code-first MVP	Single-agent útil de ponta a ponta.	Next.js; FastAPI; Postgres; LangGraph; 6-8 tools; structured outputs; HITL; audit trail; Docker.
2. RAG + context	Agente usa conhecimento de produto/ICP com contexto controlado.	pgvector; ingestion; retrieval; reranking opcional; context builder; retrieval evals.
3. Production AI Engineering	Qualidade mensurável e operação segura.	LangSmith; OpenTelemetry; Sentry; eval suite; feedback loop; prompt injection tests; RBAC; PII controls.
4. Async + scale	Processar volume e tarefas long-running de forma resiliente.	Redis; Celery; retries/backoff; DLQ/failed jobs; rate-limit coordination; scheduled jobs.
5. Cloud + CI/CD	Deploy reproduzível e versionado.	GitHub Actions; Terraform; AWS; Vercel; staging/prod; rollback.
6. Multi-agent (opcional)	Separar responsabilidades apenas se os dados justificarem.	Supervisor + Research/Qualification/Outreach/CRM agents; evals comparando V1 vs V3.

12.1 Ordem recomendada de implementação

- Primeiro faça um vertical slice: um comando real -> context -> tool read -> reasoning -> proposed action -> HITL -> tool write -> audit trail.
- Depois adicione RAG e evals antes de aumentar autonomia.
- Só então introduza filas/workers e cloud scaling.
- Multi-agent é a última etapa, não requisito para chamar a V1 de concluída.

13. Critérios de aceite e definição de pronto

13.1 Definition of Done do MVP

- Usuário autenticado consegue consultar e priorizar accounts por linguagem natural.
- O agente usa pelo menos 5 tools reais com schemas validados.
- Ações de escrita de maior risco passam por HITL com Approve/Edit/Reject.
- Todo agent run e tool call gera audit trail consultável na UI.
- O agente persiste e retoma state após approval.
- Existe uma suíte mínima de evals com baseline registrado.
- Demo público usa tenant sintético e não expõe credenciais ou dados reais.
- Repositório sobe localmente com Docker Compose e possui README arquitetural.

13.2 Definition of Done da versão de portfólio forte (V2/V5)

- RAG com retrieval metrics e corpus versionado.
- LangSmith + OpenTelemetry + Sentry em operação.
- Redis/Celery com retries, failed jobs e observabilidade de queue/worker.
- CI bloqueia merge/deploy quando tests ou eval thresholds falham.
- Deploy versionado com rollback demonstrável.
- RBAC, least privilege, prompt injection suite e PII controls documentados.
- Dashboard mostra task success, latency, cost/run, tool failure e feedback.
- Frontend público na Vercel e backend production-style em cloud.

14. Deploy público, feedback e entregáveis de portfólio

14.1 Arquitetura de demo

Componente	Hospedagem recomendada
Next.js UI	Vercel
FastAPI + LangGraph	AWS ECS/Fargate
PostgreSQL + pgvector	AWS RDS PostgreSQL
Redis	AWS ElastiCache
Workers	ECS/Fargate worker service
Secrets	AWS Secrets Manager
Logs/metrics	CloudWatch + OpenTelemetry
AI traces/evals	LangSmith

Demo Mode obrigatório

O demo público deve usar tenant sintético, quotas rígidas, rate limits e tools sandboxadas. Email/calendar externos devem enviar apenas para destinos seguros ou serem mockados. O objetivo é demonstrar comportamento, não permitir automação irrestrita por visitantes.

14.2 Feedback in-product

- Rating 1-5 e comentário opcional ligados a agent_run_id.
- Marcar se a resposta foi útil, se a ação proposta estava correta e se o usuário precisou editar/rejeitar.
- Usar feedback como fonte de casos para a regression suite, após triagem humana.

14.3 Entregáveis públicos

- URL pública do demo.
- Repositório GitHub com README, arquitetura, setup local e decisões técnicas.
- Vídeo curto mostrando um agent run completo, HITL e Agent Activity.
- Relatório de evals com baseline e evolução de versões.
- Diagrama de arquitetura e threat/risk model resumido.
- Post técnico explicando por que a solução migrou de low-code para code-first e quais trade-offs apareceram.

15. Matriz de conceitos de AI Engineering aplicados

Conceito / prática	Aplicação concreta no projeto
Agentic workflows	LangGraph com nodes, edges, routing e tools em tarefas multi-step.
Stateful agents	Checkpoints persistentes e retomada após interrupções/HITL.
Tool calling	CRM, email, calendar, research e analytics como tools reais.
Structured outputs	Pydantic/JSON Schema para scoring, classifications e tool arguments.
Context engineering	Context builder por tarefa, token budget, seleção de business state e RAG.
Prompt engineering	Prompts versionados, few-shot, policies e regression tests.
AI steering	Graph routing, system policies, tool allowlists, risk rules e limits.
RAG	ICP/playbooks/product docs em pgvector.
Hybrid retrieval/reranking	Melhorar recuperação antes de geração; medir separadamente.
Memory	Thread state, business state e long-term knowledge separados.
HITL	Interrupt/checkpoint antes de external writes sensíveis.
Least privilege	Scopes mínimos por tool, agent, environment e role.
Output validation	Schema + domain rules + authorization antes de ação.
MCP	MCP server para expor tools de CRM/RevOps com contratos claros.
Multi-agent	Supervisor e especialistas apenas na fase avançada.
Evals	Golden datasets, task success, classification, tool choice, RAG e safety.
LLM-as-judge	Complemento para critérios subjetivos com validações de consistência.
Regression testing	Reexecutar evals após mudança de prompt/model/graph.
Observability	LangSmith traces + app telemetry + Agent Activity.
OpenTelemetry	Spans distribuídos entre API, graph, workers e tools.
Reliability	Timeout, retry/backoff, rate limits, idempotency, safe reprocessing.
Dead-letter handling	Persistir jobs irrecuperáveis e permitir retry controlado.
Async processing	asyncio + Redis/Celery para background/long-running jobs.
Horizontal scaling	Workers escalados conforme queue length/latency/throughput.
Cost engineering	Tokens/run, cost/run, caching e model routing.
Latency engineering	Parallel I/O, caching, async jobs e tracing de gargalos.
Model routing/fallback	Provider/model por complexidade, custo e disponibilidade.
Prompt injection defense	Data/instruction separation, policy boundaries e attack evals.
PII protection	Redaction em traces/evals e tenant isolation.
RBAC	Roles e permissões para leitura/escrita/approvals.
Auditability	agent_runs + agent_actions + approvals + versões.
CI/CD	Tests + evals + security -> build -> staging -> prod.
Eval gate	Bloquear deploy se qualidade cair abaixo de thresholds.
Versioning	Código, prompt, graph, model, RAG e dataset versionados.
Rollback	Retornar a Docker/prompt/graph version anterior.
Infrastructure as Code	Terraform para cloud reproduzível.
LLMOps	Deploy -> traces -> feedback -> eval dataset -> melhoria.

16. Estrutura de repositório sugerida

```
agentic-revops-platform/  
  apps/  
    web/           # Next.js  
    api/           # FastAPI  
  agent/  
    graph/         # LangGraph graph + nodes  
    prompts/       # versioned prompts  
    context/       # context builders  
    policies/      # guardrails / risk rules  
    tools/         # tool contracts + implementations  
  workers/        # Celery tasks  
  data/  
    models/        # SQLAlchemy  
    migrations/    # Alembic  
    rag/           # ingestion / retrieval  
  evals/  
    datasets/  
    scorers/  
    regression/  
  tests/  
    unit/  
    integration/  
    adversarial/  
  infra/  
    docker/  
    terraform/  
  docs/  
    architecture/  
    decisions/  
  .github/workflows/
```

16.1 Narrativa de portfólio desejada

Pitch final

"I originally solved this GTM problem in production using n8n for a UK company. I then rebuilt the domain from the ground up as a production-style agentic AI platform in Python using FastAPI and LangGraph, with stateful agents, RAG, MCP tools, human-in-the-loop approvals, structured tool execution, eval-driven development, distributed tracing, asynchronous workers, Redis, PostgreSQL, AWS, CI/CD, security guardrails and versioned deployments."

16.2 Métricas que você poderá publicar do novo projeto

- Offline eval scores por versão.
- Task success rate.
- Tool selection / argument validity.
- RAG retrieval quality.
- Prompt injection resistance em suíte adversarial.
- Latency p50/p95.
- Tokens/run e cost/run.
- Worker throughput e queue behavior em load test.
- Human approval/edit/reject rate no demo.
- User feedback ratings.

Separação importante

As métricas originais da SociallyMe servem como credencial de domínio. As métricas do novo projeto devem ser apresentadas como benchmarks/evals do sistema de portfólio, não como resultados comerciais do cliente original.

17. Checklist de construção

- ☐ Definir entidades e ERD
- ☐ Criar repo monorepo e Docker Compose
- ☐ Subir PostgreSQL e migrations
- ☐ Implementar FastAPI + auth básica
- ☐ Implementar LangGraph single-agent + checkpointing
- ☐ Criar 5-8 tools com Pydantic schemas
- ☐ Criar policy layer e risk matrix
- ☐ Implementar HITL Approve/Edit/Reject
- ☐ Persistir agent_runs / agent_actions / approvals
- ☐ Criar frontend CRM + AI Assistant + Agent Activity
- ☐ Adicionar RAG + pgvector + ingestion versionada
- ☐ Criar context builder e token budget
- ☐ Criar golden eval dataset e scorers
- ☐ Adicionar LangSmith + OpenTelemetry + Sentry
- ☐ Adicionar Redis + Celery + retries + failed jobs
- ☐ Adicionar MCP server
- ☐ Adicionar prompt injection / PII tests
- ☐ Adicionar CI quality + eval gates
- ☐ Containerizar e versionar por Git SHA
- ☐ Provisionar AWS com Terraform
- ☐ Publicar Next.js na Vercel
- ☐ Criar demo tenant sintético e quotas
- ☐ Adicionar feedback in-product
- ☐ Publicar README, architecture docs e eval report
- ☐ Gravar vídeo demo
- ☐ Avaliar se multi-agent melhora resultados antes de implementar

18. Base factual e decisões herdadas

Este guia consolida o projeto a partir de três fontes fornecidas e da definição arquitetural feita nesta conversa:

- Currículo atual: especialmente SociallyMe (sistema B2B outbound com n8n/Airtable/Apollo/Instantly/GPT-4o), além de FleetPay, Alexandria, Home Carers e Tintas Farben como experiências complementares.
- Nota "Melhor sugestão projeto Agentic AI": proposta de Agentic CRM & Operations Platform com LangGraph, tools, human-in-the-loop, agent_actions/audit trail e evolução em fases.
- Nota "Recomendação stack Agentic AI Engineer": Python, FastAPI/Pydantic/SQLAlchemy/Alembic/httpx, LangGraph/LangChain, MCP, PostgreSQL/pgvector, Redis, Celery/Dramatiq, Next.js, Docker, GitHub Actions e LangSmith.

18.1 Decisões finais do guia

- Especializar o CRM genérico em GTM/Revenue Operations para aproveitar experiência real de domínio.
- Usar LangGraph como framework principal; evitar framework stacking artificial.
- Adicionar segurança de agent tools, evals, context engineering, OpenTelemetry, CI eval gates, AWS e Terraform para elevar o sinal de AI Engineer.
- Frontend público na Vercel; backend final production-style em AWS.
- Single-agent primeiro; multi-agent somente após evidência de benefício.

Critério de sucesso do projeto

Ao final, o repositório e o demo devem permitir que um entrevistador veja não apenas que você sabe chamar um LLM, mas que consegue projetar, medir, proteger, operar, escalar, versionar e melhorar um sistema agentic de ponta a ponta.